

AI 模拟面试与能力提升平台

项目设计文档

简历驱动 · RAG 可解释检索 · LLM 动态追问 · 规则降级 · 五维度评分

项目名称	AI 模拟面试与能力提升平台
专业	计算机科学与技术
队长姓名	金帅然

代码仓库: <https://github.com/JIN-jsr/ai-interview-platform>

目录

正文页码自 1 起；封面与目录不计入正文页数。

1. 项目背景与目标	1
2. 需求分析	1
3. 系统总体架构	2
4. 技术栈与运行环境	2
5. 简历解析与候选人画像	3
6. 五岗位能力模型	4
7. RAG 知识库来源与构建	4
8. LLM、RAG 与规则 fallback 协同机制	5
9. 面试流程与项目深挖调度	5
10. 回答分析与评分校准	5
11. 评分报告与能力成长	6
12. 界面与交互设计	7
13. 安全、隐私与 API Key 管理	8
14. 系统测试与运行结果	9
15. AI 辅助开发与人工审查	9
16. 创新点、局限性与后续展望	10

阅读提示 正文主要呈现系统架构、RAG 构建、项目深挖、评分校准、运行测试和 AI 辅助开发；详细操作与检查清单见仓库 docs/ 目录。

1. 项目背景与目标

本项目面向计算机相关专业学生和求职者，构建了一个由简历驱动的模拟面试与能力提升平台。系统不能代替真实面试官，而是帮助用户围绕简历、目标岗位和项目经历进行训练、复盘与提升。

平台强调本地可运行、过程可解释、LLM 异常可降级、评分依据可追溯，形成“简历解析—候选人画像—RAG 出题—连续面试—回答分析—评分报告—历史成长复盘”的完整闭环。

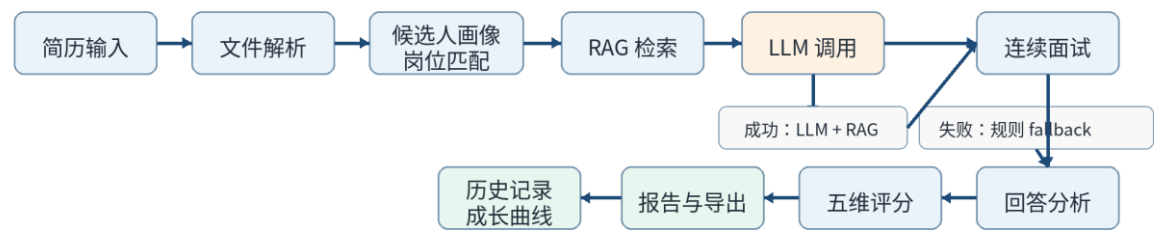
2. 需求分析

- 输入与画像：支持简历文本、TXT/PDF/DOCX 上传、多文件合并、结构化解析与岗位匹配提醒。
- 面试过程：支持岗位和难度选择、RAG 检索、LLM 动态问题、项目深挖、上下文追问与规则备用出题。
- 结果反馈：输出五维度评分、评分依据、薄弱点、学习建议、简历优化建议、历史记录和能力成长曲线。
- 稳定性要求：在网络超时、模型权限不足或无 API Key 时，在 USE_LLM=false 模式下，核心流程仍可由本地规则继续运行。

使用边界 评分与建议用于学习训练、自我复盘和面试准备，不应作为真实招聘录用或个人能力认定的唯一依据。

3. 系统总体架构

系统总体架构与数据流



核心原则：RAG 提供可追溯知识依据；LLM 负责自然表达；异常时自动降级；评分由本地规则计算并保留证据。

图 1 系统总体架构与核心数据流

简历输入经解析模块生成候选人画像和岗位匹配信息；RAG 模块根据岗位、技能关键词、难度及已用知识点检索问题依据；LLM 可用时负责自然表达，失败时由本地规则 fallback 接管；回答经分析与评分后生成结构化报告，并由会话管理模块保存历史记录。各模块通过结构化数据传递候选人画像、问题元数据、回答记录和评分证据，使出题、分析、报告与历史会话之间保持一致并便于追溯。

4. 技术栈与运行环境

类别	技术与用途
Web 框架	Streamlit: 本地交互式 Web 界面
开发语言	Python 3.12
知识库	本地 JSON 文件
LLM 模型与接口	阿里云百炼 OpenAI 兼容 Chat Completions API (qwen3.7-plus)
简历读取	pdfplumber、python-docx、TXT 文本读取
配置与请求	python-dotenv、requests
报告导出	Pillow; LaTeX/XeLaTeX 生成正式 PDF

表 1 主要技术栈与用途

本地验证环境为 Windows、Python 3.12 和 Streamlit；详细安装与启动命令以仓库 README 为准。

5. 简历解析与候选人画像

系统支持粘贴简历文本，也支持 TXT、PDF、DOCX 文件上传及多文件合并。解析优先尝试 LLM，失败时使用本地规则提取基本信息、技能、项目、面试重点与潜在薄弱项；岗位匹配模块结合目标岗位和项目描述给出温和提醒与优化建议。

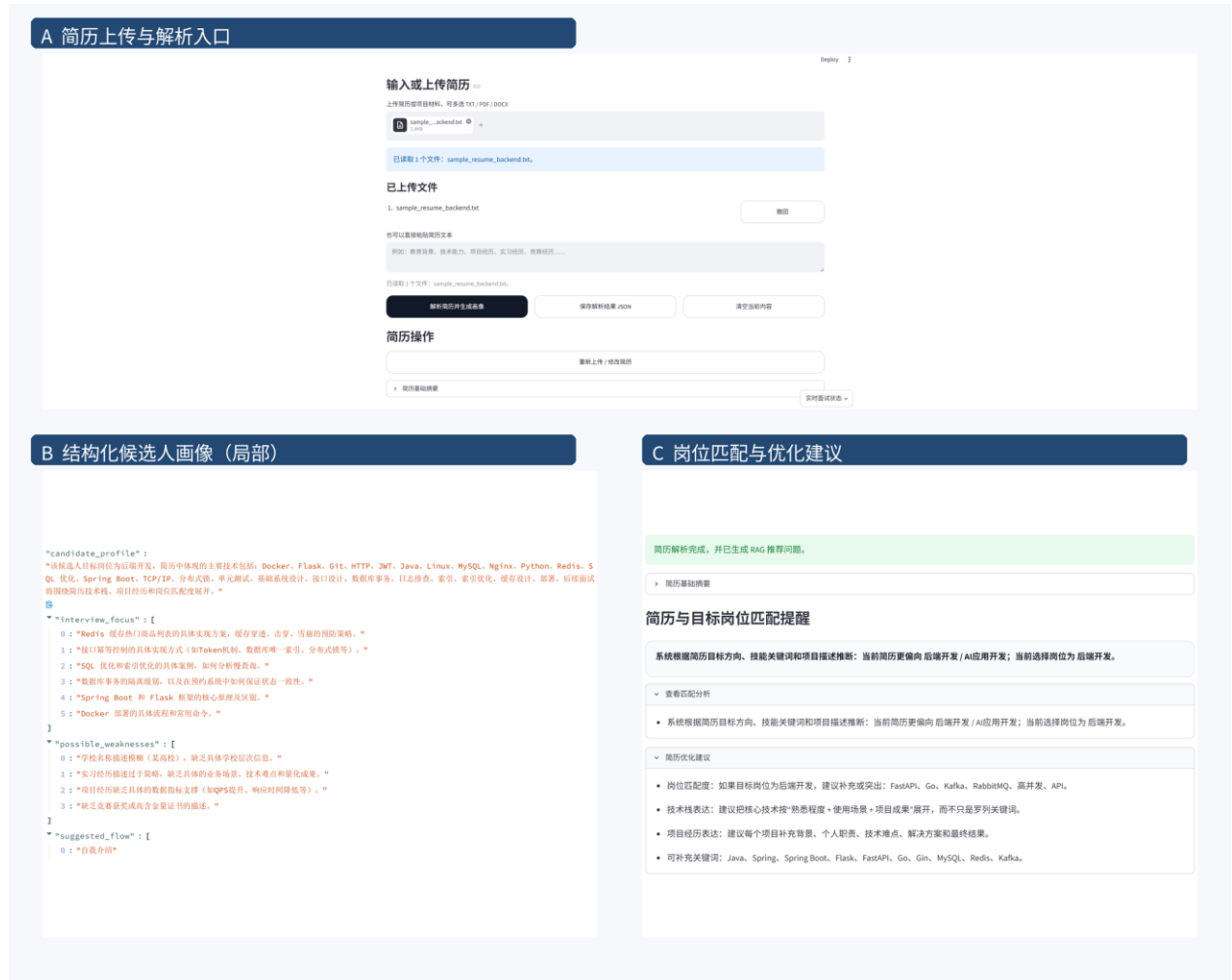


图 2 简历输入、结构化画像与岗位匹配分析

设计要点 画像包含技能、项目经历、面试重点和潜在薄弱项，并作为 RAG 检索、岗位能力覆盖、项目深挖和简历优化的共同输入；原始结构化结果可导出为 JSON，界面则优先展示可读摘要。

6. 五岗位能力模型

系统支持后端开发、前端开发、AI 应用开发、数据分析和软件测试五个目标岗位。能力词库统一维护在 `src/product_features.py`，RAG 检索与报告能力覆盖共用同一来源，减少多处配置不一致。

岗位	能力覆盖重点
后端开发	Java/Python/Go、Spring/FastAPI/Gin、数据库、缓存、消息队列、高并发、微服务、部署与监控
前端开发	HTML/CSS/JavaScript/TypeScript、Vue/React、组件化、状态管理、跨域、性能、安全与 E2E 测试
AI 应用开发	LLM API、Prompt、RAG、结构化输出、Token 管理、fallback、幻觉控制与工具调用
数据分析	SQL、Pandas、数据清洗、指标体系、A/B 测试、漏斗留存、统计显著性与结论表达
软件测试	用例设计、接口/自动化/性能测试、pytest、Mock、回归、Flaky Test 与 LLM/RAG 测试

表 2 五岗位能力覆盖重点

7. RAG 知识库来源与构建

知识库位于 `data/knowledge_base.json`，内容参考课程知识、教材、官方文档及可靠公开面试资料，经人工筛选、去重、分类和校验后整理为 JSON 条目。当前共 330 条，涵盖问题、参考答案、期望要点、岗位标签、难度及常见误区，覆盖五个目标岗位，并补充项目深挖、故障排查和测试验证内容。



图 3 RAG 出题依据与期望回答要点示例

系统采用本地可解释关键词检索，根据目标岗位、简历技能、项目描述、难度及已用知识点选择问题依据，并避免短时间重复出题。当前未使用向量数据库、Embedding、BM25 或语义重排模型，便于本地运行和人工核验。

8. LLM、RAG 与规则 fallback 协同机制

- RAG 决定考查内容和方向：提供知识点、参考答案、期望要点、难度与误区依据。
- LLM 负责将检索结果转化为自然问题，并结合上下文生成连续追问：结合画像、最近对话和 RAG 条目生成连续问题，返回结构化 JSON，并反馈文字生成；知识点编号、期望要点、评分结果及证据标记均由本地逻辑保留和校验。
- 规则 fallback 保证稳定：当 LLM 未启用、超时、格式异常或重复出题时，自动切换至本地模板并记录原因。

9. 面试流程与项目深挖调度

完整面试约 8 题，目标结构包括开场题、RAG 知识或工程题、至少 2 道项目深挖题、至少 1 道上下文追问及可选综合岗位题。调度逻辑会在接近结束时修复项目深挖或追问覆盖不足。上下文追问会从上一轮回答中提取关键技术点、遗漏内容或表述矛盾，继续追问原理、实现细节、方案取舍与验证方式。

证据不足处理 若简历项目证据不足，系统不会编造项目细节，而是标记 `project_evidence_sufficient=false`、`project_evidence_confidence=low`，并提示该维度评价可信度较低。

10. 回答分析与评分校准

每次提交回答后，系统依据 `expected_points`、参考答案、技术关键词及常见误区，通过 LLM 辅助生成轻量即时分析，展示已覆盖要点、主要缺失和改进方向；值得注意的是，系统支持 LLM 辅助润色每题即时反馈和总体回答分析，不修改覆盖率、临时分以及最终分数等；当 LLM 不可用时自动回退本地规则。最终五星评分仍在面试结束后基于全部回答统一计算，权重保持不变：

维度	权重
基础知识掌握程度	25%
项目理解深度	25%
回答逻辑性	20%
表达完整性	15%
岗位匹配度	15%

表 3 五维度评分权重

- 不因关键词堆砌或结构词给出高分；逻辑满分仅用于多轮稳定且论证严密的回答。
- 长篇重复回答受到信息密度与简洁性限制；明确技术误区会影响技术得分。
- 项目题不足时降低评价置信度，而不是把调度缺陷隐藏为候选人能力不足。

11. 评分报告与能力成长

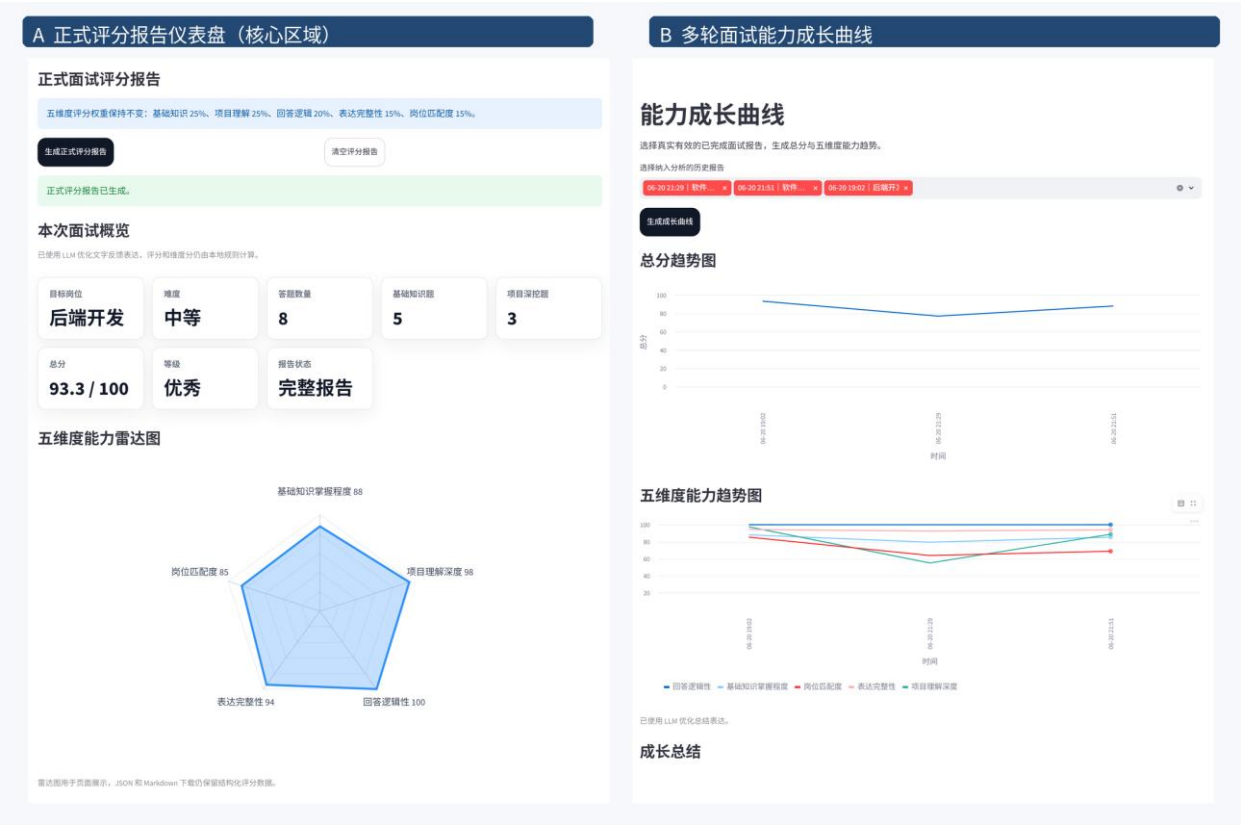


图 4 正式评分报告核心区域与多轮面试能力成长曲线

报告包含总分、等级、五维评分、评分置信度、问题分布、回答稳定性、岗位能力覆盖、薄弱点卡片、学习建议和简历优化建议。LLM 仅负责润色文字反馈，不得修改分数、权重、题数或证据标记。

- 历史会话保存在 outputs/sessions/，可生成总分与五维度趋势。
- 报告支持 JSON、Markdown、完整长图 PNG 和摘要海报 PNG 导出。
- 完整长图用于保存细节，设计文档只展示核心仪表盘，避免图片缩小后不可读。

12. 界面与交互设计

界面保持黑白灰风格，首页提供“系统介绍”和“模拟面试”入口。左侧栏承担历史会话切换、演示模式、RAG 知识库、能力成长曲线及运行状态展示，主区域聚焦问题详情、连续面试和报告跳转。



图 5 左侧栏的面试记录、更多功能与系统状态

交互原则 答后分析以可折叠卡片呈现，默认显示简洁摘要，详细要点按需展开；用户既能及时发现问题，也不会因大段反馈打断连续面试。技术字段和完整参考依据仍通过展开区域展示。

12. 界面与交互设计（续）

 从工程落地角度看，这里还有一个关键问题。结合你提到的全表扫描和自增主键，我想深入聊聊MySQL索引。它的核心原理是什么？在实际项目中，引入索引会带来哪些收益，又有哪些容易被忽视的代价或限制？



图 6 模拟面试工作区：问题详情、进度与实时状态

工作区截图用于展示当前问题、问题类型、生成方式、难度、RAG 依据与实时状态；回答输入区位于页面下方，未在该截图中重复展示。面试结束后，系统在结束提示处提供“导出即时分析”功能和“查看报告”跳转按钮。

13. 安全、隐私与 API Key 管理

- API Key 通过 .env 管理，公开仓库仅保留 .env.example；密钥不写入代码、截图或日志。
- 真实简历、会话、报告和导出图片均属于运行数据，不应提交公开仓库。
- outputs/sessions/、outputs/reports/、outputs/report_images/ 等目录默认由 .gitignore 排除。
- 系统定位为本地学习与比赛演示工具，当前尚未实现生产级权限管理、审计和云端合规能力。

14. 系统测试与运行结果

测试项	测试结果与说明
python scripts/self_check.py	已通过：检查文件结构、图片、隐私、链接、知识库 字段与评分权重
python scripts/scoring_calibra tion_check.py	已通过：离线验证低、中、高回答梯度及误区扣分
python -m compileall app.py src scripts	已通过：Python 文件均可正常编译
RAG 知识库	已校验：知识库共 330 条
USE_LLM=false	已通过：USE_LLM=false 下可完成完整流程
USE_LLM=true	已人工验证：qwen3.7-plus 可正常调用并生成动态 追问
报告导出	已通过：四种报告格式均可正常导出

表 4 主要测试与运行验证项目

15. AI 辅助开发与人工审查

项目开发过程中使用了 ChatGPT、Codex 和 DeepSeek 等 AI 工具。ChatGPT 与 Codex 主要用于协助梳理需求、生成部分代码初稿、优化 Prompt、补充设计和测试思路以及整理文档；DeepSeek 主要用于生成用于系统测试的不同质量层次的模拟面试回答。

本人主要完成以下工作：

- 根据赛题要求确定系统整体架构、核心功能、开发顺序和评分边界，并对 AI 给出的方案进行筛选、调整、整合与不断优化；
- 统一简历解析、RAG 检索、动态提问、规则 fallback、回答分析、评分报告和历史记录之间的数据流与交互逻辑；
- 持续调整页面层级、留白、字体、按钮位置、提示语，使界面兼顾比赛演示效果、信息清晰度和实际操作体验，统一页面的视觉层级、交互细节与黑白灰设计风格；

- 修复 Prompt JSON 格式冲突、Streamlit session_state 时序、LLM 超时与 fallback 等问题，并核对 RAG 条目、展示字段、评分权重、报告依据和导出排版；
- 使用不同质量的模拟回答开展日常测试和评分校准，检查关键词堆砌、回答篇幅和结构化表达造成的分数膨胀，并进行针对性优化；
- 检查 API Key、真实简历、会话记录和报告等隐私数据，确保公开仓库不包含敏感信息。

本人能够说明简历解析、候选人画像、RAG 检索、LLM 动态出题、规则降级、回答分析、多维评分、报告生成和会话管理的主要数据流。AI 在本项目中承担辅助生成代码、辅助方案设计和测试支持的角色，最终的功能取舍、界面设计、代码整合、测试验证和质量审查均由本人完成。

16. 创新点、局限性与后续展望

创新点

- 系统将简历信息、RAG 知识条目、问题元数据、回答覆盖情况和评分依据关联起来，形成可追溯的面试评价证据链。
- 本地可解释 RAG 能回答“为什么问这道题”，LLM 异常时 fallback 保证演示稳定。
- 评分报告同时给出证据、置信度、岗位能力覆盖、薄弱点和训练建议。

当前局限

- 检索仍以本地关键词规则为主，尚未接入真正的向量数据库与语义重排。
- 评分属于启发式训练辅助评价，不宣称完全客观，也不替代真实招聘决策。
- LLM 能力受网络、模型权限、地区 endpoint 和 API Key 影响。

后续展望

- 引入向量检索与人工标注样本，进一步校准不同岗位和题型的评分尺度。
- 扩展更多岗位模板、语音面试、云端部署、团队账号与权限管理。
- 建立更系统的人工评分基准和跨模型一致性测试。

综上，平台以“可运行、可解释、可降级、可复盘”为核心设计目标，在有限开发周期内完成了从简历解析到能力成长分析的完整功能闭环。